

lecture_21

September 26, 2026

1 Lecture 21: Transportation Engineering Problems - III

{note} Lectures 10-11 solved transportation engineering problems on a network whose nodes were *given* - the max-flow, least-cost path, transportation, and transshipment problems all decided how much flow to send, never where to put a node. Lectures 12-20 then built a complete toolkit for non-linear programs - unconstrained methods (Gradient Descent, Newton's Method, BFGS), constrained methods (Penalty, Barrier, Interior-Point), and the accompanying sensitivity and duality theory. This lecture combines both: the Facility Location Problem asks *where* to place a facility, not just how much to route through it. Because distance is a non-linear function of the facility's own coordinates, this is a genuine NLP - and it is exactly the sensitivity-analysis application Lecture 19 promised at its close.

Learning Objectives

By the end of this notebook, you will be able to: 1. Formulate a continuous single-facility location problem — combining upstream (inbound) and downstream (outbound) flow-weighted distance — as an unconstrained NLP, and as a constrained NLP under a siting restriction. 2. Solve facility-location NLPs in Python using Gradient Descent, Newton's Method, BFGS, and Interior-Point methods, and compare their behaviour on a genuinely non-linear, non-quadratic objective. 3. Use KKT multipliers and the NLP Sensitivity Theorem (Lecture 19) to quantify the cost of a siting constraint, and verify the result against the Lagrangian dual (Lecture 20).

Prerequisites: NLP Principles (Lecture 12); Gradient Descent, Newton's Method, BFGS (Lectures 13-15); Penalty, Barrier, Interior-Point Methods (Lectures 16-18); NLP Sensitivity Analysis (Lecture 19); NLP Duality (Lecture 20); Transshipment Problem (Lecture 11).

Estimated time: 90 minutes (including in-class exercises)

1.1 Why Facility Location?

Lecture 12's second motivating example previewed this problem: a logistics firm wants to site a hub at coordinates (x, y) to minimize total weighted distance to n customers. Real siting decisions are rarely one-sided — a consolidation center typically receives goods from a handful of upstream sources (plants, ports, supplier depots) and dispatches them to a larger set of downstream destinations (retail clusters, customer zones). This lecture asks:

- Delhivery asks: *where in the Chennai metropolitan region should we site a new regional consolidation center, given inbound tonnage from our supplier plants and outbound tonnage to our retail clusters, so that total daily transport cost is minimized?* — an unconstrained Facility Location Problem.
- The same question, but civic authorities or service-level agreements (SLAs) restrict *where* the facility may go — e.g. it must stay within a maximum response distance of a flagship store, or outside an environmentally protected buffer — a constrained Facility Location Problem.

Both questions route goods by direct trips only (no intermediate hubs, no vehicle routes — that is the subject of a later course on logistics network design). What makes them genuinely non-linear is that the decision variable is the facility's own location, and the cost of serving each source or destination is proportional to the *Euclidean distance* to that location — a non-linear function of (x, y) .

{note} Contrast this with Lecture 11's Transshipment Problem: there, the hub's *location* was fixed and only the *flow* through it was a decision variable, so the LP toolkit sufficed. Here, the hub's *location itself* is the decision variable, the objective is a sum of square roots, and no amount of network structure turns this back into a linear program. This is precisely the kind of problem Lectures 13-20 were built for.

1.2 Notation

Symbol	Meaning
(x, y)	Decision variables — coordinates of the facility to be sited (km, local planar grid)
p	Number of upstream source nodes (plants, supplier depots)
$(a_k, b_k), k = 1, \dots, p$	Coordinates of upstream source k
u_k	Inbound flow from source k to the facility (tonnes/day)
q	Number of downstream destination nodes (retail clusters)
$(c_l, d_l), l = 1, \dots, q$	Coordinates of downstream destination l
v_l	Outbound flow from the facility to destination l (tonnes/day)
$\text{dist}_k(x, y) = \sqrt{(x - a_k)^2 + (y - b_k)^2}$	Euclidean distance from the facility to source k
$\text{dist}_l(x, y) = \sqrt{(x - c_l)^2 + (y - d_l)^2}$	Euclidean distance from the facility to destination l
κ	Transport cost rate (per tonne-km)

1.3 The Unconstrained Facility Location Problem (Weber Problem)

1.3.1 Problem Statement

Given p upstream sources with inbound flows u_k and q downstream destinations with outbound flows v_l , find the facility location (x, y) that minimizes total flow-weighted distance — equivalently, total daily transport cost at rate κ per tonne-km.

1.3.2 NLP Formulation

Decision variables: $x, y \in \mathbb{R}$ — the facility's coordinates.

Objective: minimize total flow-weighted distance,

$$\min_{x,y} f(x, y) = \sum_{k=1}^p u_k \text{dist}_k(x, y) + \sum_{l=1}^q v_l \text{dist}_l(x, y)$$

Subject to: none — this is the unconstrained case, $(x, y) \in \mathbb{R}^2$.

Gradient:

$$\nabla f(x, y) = \left[\begin{array}{c} \sum_k u_k \frac{x - a_k}{\text{dist}_k(x, y)} + \sum_l v_l \frac{x - c_l}{\text{dist}_l(x, y)} \\ \sum_k u_k \frac{y - b_k}{\text{dist}_k(x, y)} + \sum_l v_l \frac{y - d_l}{\text{dist}_l(x, y)} \end{array} \right]$$

{note} Each term $\sqrt{(x-a_k)^2+(y-b_k)^2}$ is convex (it is a norm), and a non-negatively weighted sum of convex functions is convex — so $f(x,y)$ is convex and any stationary point $\nabla f = \mathbf{0}$ is the global minimum (Lecture 12's Taxonomy). The one wrinkle: f is not differentiable exactly at any source or destination point, since $\text{dist}_k \rightarrow 0$ makes the gradient term blow up. Gradient-based methods are safe as long as no iterate lands exactly on a data point — true almost always in practice, and true throughout this lecture.

1.4 In-Class Exercises

1.4.1 Exercise 1 — Unconstrained Facility Location

Problem Statement Delhivery wants to site a new regional consolidation center in the Chennai metropolitan region. The facility will receive inbound freight from two supplier plants and dispatch outbound freight to three retail clusters (all coordinates in km, on a local planar grid; transport cost $\kappa = 40/\text{tonne-km}$):

Node	Type	x (km)	y (km)	Flow (tonnes/day)
Ambattur Industrial Estate (S_1)	Upstream	10	40	30
Oragadam Auto-Component Cluster (S_2)	Upstream	60	10	20
Tambaram Retail Hub (D_1)	Downstream	20	5	15
Anna Nagar Retail Hub (D_2)	Downstream	50	45	20

Node	Type	x (km)	y (km)	Flow (tonnes/day)
Mahabalipuram Retail Hub (D_3)	Downstream	80	30	15

Find the facility location that minimizes total daily transport cost.

Step 1 — Formulate the NLP

$$\min_{x,y} f(x,y) = 30\sqrt{(x-10)^2 + (y-40)^2} + 20\sqrt{(x-60)^2 + (y-10)^2} \\ + 15\sqrt{(x-20)^2 + (y-5)^2} + 20\sqrt{(x-50)^2 + (y-45)^2} + 15\sqrt{(x-80)^2 + (y-30)^2}$$

This is unconstrained, convex, and non-quadratic — a genuine test of Lectures 13-15's toolkit.

Step 2 — Solve with `scipy.optimize.minimize` Start from the flow-weighted centroid $(x^{(0)}, y^{(0)}) = \left(\frac{\sum u_k a_k + \sum v_l c_l}{\sum u_k + \sum v_l}, \frac{\sum u_k b_k + \sum v_l d_l}{\sum u_k + \sum v_l} \right) = (40.00, 28.25)$, and compare BFGS (Lecture 15), CG (a Gradient-Descent-family method, Lecture 13), and Newton-CG (Lecture 14) — all supplied with the exact gradient derived above.

```
[ ]: # --- Exercise 1: unconstrained facility location (Weber problem) ---

import numpy as np
from scipy.optimize import minimize

S = np.array([[10.0, 40.0], [60.0, 10.0]])          # upstream sources
u = np.array([30.0, 20.0])                        # inbound flow (t/day)
D = np.array([[20.0, 5.0], [50.0, 45.0], [80.0, 30.0]]) # downstream
↪ destinations
v = np.array([15.0, 20.0, 15.0])                  # outbound flow (t/day)

pts = np.vstack([S, D])
w = np.concatenate([u, v])
kappa = 40 # Rs per tonne-km

def f(p):
    d = np.sqrt(((p - pts) ** 2).sum(axis=1))
    return np.sum(w * d)

def grad(p):
    diff = p - pts
    d = np.sqrt((diff ** 2).sum(axis=1))
    d = np.where(d < 1e-9, 1e-9, d)
    return (w[:, None] * diff / d[:, None]).sum(axis=0)

x0 = np.average(pts[:, 0], weights=w)
y0 = np.average(pts[:, 1], weights=w)
```

```

p0 = np.array([x0, y0])
print(f"Weighted-centroid start: ({p0[0]:.2f}, {p0[1]:.2f}), f0 = {f(p0):.2f}␣
↪tonne-km/day")

res_bfgs = minimize(f, p0, jac=grad, method='BFGS', options={'gtol': 1e-10})
res_cg = minimize(f, p0, jac=grad, method='CG', options={'gtol': 1e-10})
res_ncg = minimize(f, p0, jac=grad, method='Newton-CG', options={'xtol':␣
↪1e-10})

for name, res in [("BFGS", res_bfgs), ("CG", res_cg), ("Newton-CG", res_ncg)]:
    print(f"\n{name}: x* = ({res.x[0]:.4f}, {res.x[1]:.4f}) f* = {res.fun:.4f}␣
↪ "
           f"iterations = {res.nit} nfev = {res.nfev}")

print(f"\nDaily transport cost at optimum: Rs {res_bfgs.fun*kappa:,.0f}/day")

```

Weighted-centroid start: (40.00, 28.25), f0 = 2958.83 tonne-km/day

BFGS: x* = (41.5542, 30.2112) f* = 2953.2359 iterations = 7 nfev = 8

CG: x* = (41.5542, 30.2112) f* = 2953.2359 iterations = 5 nfev = 11

Newton-CG: x* = (41.5542, 30.2112) f* = 2953.2359 iterations = 6 nfev = 6

Daily transport cost at optimum: Rs 118,129/day

Step 3 — Method Comparison

Method (Lecture)	Direction used	Iterations	Function evals	x^*, y^*
CG (13 — Gradient-Descent family)	$d^{(k)} =$ $-\nabla f(x^{(k)})$, conjugate- corrected	5	11	(41.554, 30.211)
Newton-CG (14)	$d^{(k)} =$ $-\nabla^2 f(x^{(k)})^{-1} \nabla f(x^{(k)})$, CG-solved	6	6	(41.554, 30.211)
BFGS (15)	$d^{(k)} =$ $-[H^{(k)}]^{-1} \nabla f(x^{(k)})$, rank-2 update	7	8	(41.554, 30.211)

All three converge to the same point — expected, since f is convex (Lecture 12). A plain Gradient Descent trace from the same start, using a normalized step capped at 5 km per iteration, shows the same qualitative behaviour: a large first correction followed by rapidly shrinking steps as $\|\nabla f\| \rightarrow 0$:

k	$x^{(k)}$	$y^{(k)}$	$f(x^{(k)})$	$\ \nabla f(x^{(k)})\ $
0	40.00	28.25	2958.83	4.540
1	41.38	30.60	2953.42	0.879
2	41.58	30.11	2953.25	0.210
3	41.55	30.23	2953.24	0.049
4	41.56	30.21	2953.24	0.011
5	41.55	30.21	2953.24	0.003

{tip} Newton-CG needed the fewest function evaluations here because the Hessian gives it curvature information directly - but computing $\nabla^2 f$ costs more per iteration than BFGS's cheap rank-2 update. For a 2-variable problem like this the difference is invisible; it becomes decisive for facility-location problems with many upstream/downstream nodes and correspondingly expensive Hessians (Lecture 14's own caveat).

Step 4 — Interpretation Optimal site: $(x^*, y^*) = (41.55, 30.21)$ km, at a total flow-weighted distance of $f^* = 2953.24$ tonne-km/day — a daily transport cost of 1,18,129/day.

Managerial insight: The optimal site sits closer to Mahabalipuram (80, 30) and Anna Nagar (50, 45) than to either upstream plant, because their combined outbound flow (35 t/day) exceeds the flow from either single plant. Delivery does not need to weight the two supplier plants equally with the three retail clusters — the Weber solution automatically pulls the site toward whichever side of the network carries more tonnage, exactly as a weighted center of mass would.

1.5 The Constrained Facility Location Problem (Siting / Coverage Restrictions)

1.5.1 Problem Statement

Real siting decisions are rarely free of restrictions. Two common types recur in practice:

- Coverage / SLA constraint: the facility must lie within a maximum response distance R of a demand point that has a strict service-level agreement — $g(x, y) = \text{dist}(x, y)^2 - R^2 \leq 0$.
- Exclusion / keep-out constraint: the facility must stay outside a protected buffer (residential zone, eco-sensitive area) of radius R around some point — $g(x, y) = R^2 - \text{dist}(x, y)^2 \leq 0$.

Both are single non-linear inequality constraints on the same decision variables (x, y) ; the KKT machinery of Lectures 16-19 applies unchanged.

1.5.2 NLP Formulation (Coverage Case)

$$\min_{x,y} f(x, y) \quad \text{subject to} \quad g(x, y) = (x - x_0)^2 + (y - y_0)^2 - R^2 \leq 0$$

where (x_0, y_0) is the SLA-critical demand point and R is the maximum permitted response distance.

{note} Unlike the quadratic examples reused throughout Lectures 16-20 (MTC feeder cost, NHAI parallel links), the Weber objective $f(x, y)$ has no closed form for

its unconstrained minimizer, so the constrained problem cannot be solved by substitution either. Everything here is done numerically - this is the typical situation in practice, and exactly why Lectures 16-18 built general-purpose iterative algorithms rather than relying on closed-form KKT solves.

1.5.3 Exercise 2 — Constrained Facility Location with Sensitivity Analysis

Problem Statement Mahabalipuram Retail Hub (80, 30) is a flagship same-day-delivery store: Delhivery's SLA requires the consolidation center to lie within $R = 30$ km of it. All other data are unchanged from Exercise 1.

Step 1 — Formulate the NLP

$$\min_{x,y} f(x,y) \quad (\text{as in Exercise 1}) \quad \text{subject to} \quad g(x,y) = (x-80)^2 + (y-30)^2 - 900 \leq 0$$

Step 2 — Solve with `scipy.optimize.minimize(method='trust-constr')` The Interior-Point method (Lecture 18) is the natural choice here, since `trust-constr` both handles the non-linear constraint directly and returns the KKT multiplier as a by-product.

```
[ ]: # --- Exercise 2: constrained facility location (SLA coverage radius) ---

from scipy.optimize import NonlinearConstraint

target = np.array([80.0, 30.0]) # Mahabalipuram Retail Hub
R = 30.0 # SLA response-distance limit (km)

def g(p):
    return ((p - target) ** 2).sum() # <= R2

nlc = NonlinearConstraint(g, -np.inf, R2)
res_c = minimize(f, res_bfgs.x, jac=grad, method='trust-constr',
                 constraints=[nlc], options={'gtol': 1e-12, 'xtol': 1e-14})

print(f"Constrained optimum: x* = ({res_c.x[0]:.4f}, {res_c.x[1]:.4f}) f* = {res_c.fun:.4f}")
print(f"Constraint value g(x*) = {g(res_c.x):.4f} (R^2 = {R2:.1f}) -> active")
lam = res_c.v[0][0]
print(f"KKT multiplier lambda* (w.r.t. b = R^2) = {lam:.5f}")

print(f"\nDaily transport cost at constrained optimum: Rs {res_c.fun*kappa:,.0f}/day")
print(f"Cost of the SLA constraint: Rs {(res_c.fun - res_bfgs.fun)*kappa:,.0f}/day "
      f"(vs. unconstrained Rs {res_bfgs.fun*kappa:,.0f}/day)")
```

Constrained optimum: $x^* = (50.0002, 29.8997)$ $f^* = 3019.9032$
 Constraint value $g(x^*) = 900.0000$ ($R^2 = 900.0$) $\rightarrow$ active
 KKT multiplier λ^* (w.r.t. $b = R^2$) = 0.27749

Daily transport cost at constrained optimum: Rs 120,796/day
 Cost of the SLA constraint: Rs 26,667/day (vs. unconstrained Rs 118,129/day)

The SLA pulls the facility from (41.55, 30.21) to (50.00, 29.90) — directly toward Mahabalipuram — at an extra cost of 2,667/day ($2,667 = 66.67 \text{ tonne-km/day} \times 40/\text{tonne-km}$).

Step 3 — Sensitivity Analysis: How Much Would Relaxing the SLA Save? This is exactly the question Lecture 19 promised. Following the NLP Sensitivity Theorem, parametrize the constraint as $g(x, y) \leq b$ with $b = R^2$: $\lambda^* = -\partial f^*/\partial b$. Because f has no closed form here, we verify the theorem numerically rather than symbolically — re-solve the constrained NLP at $b_0 \pm \Delta b$ and take a central difference.

```
[ ]: # --- Exercise 2: verifying the Sensitivity Theorem numerically ---

def solve_at_R(Rval, x0=res_bfgs.x):
    nlc_r = NonlinearConstraint(g, -np.inf, Rval2)
    return minimize(f, x0, jac=grad, method='trust-constr',
                    constraints=[nlc_r], options={'gtol': 1e-12, 'xtol': 1e-14})

b0 = R**2
db = 1.0
res_p = solve_at_R(np.sqrt(b0 + db))
res_m = solve_at_R(np.sqrt(b0 - db))
dfdb_fd = (res_p.fun - res_m.fun) / (2 * db)
print(f"Finite-difference df*/db = {dfdb_fd:.5f} => -df*/db = {-dfdb_fd:.5f}")
print(f"KKT multiplier lambda* = {lam:.5f} (matches within numerical tolerance)")

# more intuitive: sensitivity directly with respect to R (not R^2)
res_Rp = solve_at_R(R + 1.0)
res_Rm = solve_at_R(R - 1.0)
dfdR_fd = (res_Rp.fun - res_Rm.fun) / 2.0
lam_R = -dfdR_fd
print(f"\nf*(R=29) = {res_Rm.fun:.4f}    f*(R=30) = {res_c.fun:.4f}    f*(R=31) = {res_Rp.fun:.4f}")
print(f"lambda_R = -df*/dR = {lam_R:.4f} tonne-km/day per km "
      f"(chain rule check: lambda*.2R = {lam*2*R:.4f})")
print(f"Value of relaxing the SLA by 1 km: Rs {lam_R*kappa:,.0f}/day")
```

Finite-difference $df^*/db = -0.27749 \Rightarrow -df^*/db = 0.27749$
 KKT multiplier $\lambda^* = 0.27749$ (matches within numerical tolerance)

$f^*(R=29) = 3037.6480$ $f^*(R=30) = 3019.9032$ $f^*(R=31) = 3004.3476$

$\lambda_R = -df^*/dR = 16.6502$ tonne-km/day per km (chain rule check: $\lambda \cdot 2R = 16.6496$)

Value of relaxing the SLA by 1 km: Rs 666/day

Valid range of this result. Unlike the quadratic examples in Lecture 19, where the shadow price is *constant* across a ranging interval, λ_R here changes continuously with R — a direct consequence of f being convex but non-quadratic. Re-solving across a grid of R values traces out how the SLA's marginal cost decays:

R (km)	f^* (tonne-km/day)	$\lambda(R)$ (tonne-km/day per km)
10	3756.98	2.665
15	3505.99	1.557
20	3293.88	0.945
25	3130.37	0.550
30	3019.90	0.277
35	2963.48	0.088
38	2953.40	0.010
38.45	2953.24	≈ 0

{caution} The constraint is only active - and only then does $\lambda(R) > 0$ - while the SLA genuinely binds. At $R \approx 38.45$ km (the unconstrained optimum's own distance to Mahabalipuram), $\lambda(R) \rightarrow 0$: relaxing the SLA further changes nothing, because the unconstrained optimum from Exercise 1 already satisfies it. This is the primal-feasibility mechanism from Lecture 19's ranging procedure - the active-set assumption breaks down exactly where $\lambda(R)$ hits zero, giving the valid range $R \in (0, 38.45]$ km for this constraint to be meaningfully binding.

Step 4 — Interpretation

Managerial insight: Near $R = 30$ km, every additional kilometre of SLA slack is worth about 666/day to Delhivery — but this marginal value shrinks as R grows, from 107/day per km near $R = 10$ down to nearly zero by $R = 38$ km. A one-time relaxation from 30 km to 35 km would save roughly $5 \times \bar{\lambda}_R \approx 2,300$ /day (using the average λ_R over that range) — a concrete number Delhivery's operations team can weigh against the cost of negotiating a looser SLA with the Mahabalipuram store.

1.5.4 Bonus: Verifying Strong Duality

Lecture 20 showed that λ^* is simultaneously the KKT multiplier *and* the solution to the Lagrangian dual $\max_{\lambda \geq 0} q(\lambda)$, with $q(\lambda) = \inf_{x,y} \mathcal{L}(x,y,\lambda)$. Because f is convex and a Slater point trivially exists (any point with $g(x,y) < 0$, e.g. Mahabalipuram itself), strong duality should hold exactly.

```
[1]: # --- Verifying strong duality: dual function q(lambda) ---
```

```

def L(p, lam):
    return f(p) + lam * g(p)

def gradL(p, lam):
    diff = p - target
    return grad(p) + lam * 2 * diff

def q(lam, x0=res_c.x):
    res = minimize(L, x0, args=(lam,), jac=gradL, method='BFGS',
    ↪options={'gtol': 1e-12})
    return res.fun

for lam_test in [0.0, 0.10, 0.20, 0.25, 0.27, 0.2775, 0.28, 0.30, 0.35, 0.50]:
    print(f"  lambda = {lam_test:.4f}:  q(lambda) = {q(lam_test):.4f}")

print(f"\nq(lambda*) = {q(0.2775):.4f}")
print(f"f*(primal, R=30) = {res_c.fun:.4f}")
print("Strong duality holds: q(lambda*) = f* to 4 decimal places.")

```

```

lambda = 0.0000:  q(lambda) = 2953.2359
lambda = 0.1000:  q(lambda) = 2995.8058
lambda = 0.2000:  q(lambda) = 3015.7196
lambda = 0.2500:  q(lambda) = 3019.3973
lambda = 0.2700:  q(lambda) = 3019.8662
lambda = 0.2775:  q(lambda) = 3019.9032
lambda = 0.2800:  q(lambda) = 3019.8991
lambda = 0.3000:  q(lambda) = 3019.5765
lambda = 0.3500:  q(lambda) = 3016.6286
lambda = 0.5000:  q(lambda) = 2991.8866

```

```
q(lambda*) = 3019.9032
```

```
f*(primal, R=30) = 3019.9032
```

```
Strong duality holds: q(lambda*) = f* to 4 decimal places.
```

$q(\lambda)$ is concave in λ (Lecture 20, Property 1), peaks exactly at $\lambda^* = 0.2775$, and $q(\lambda^*) = f^* = 3019.9032$ — a zero duality gap, confirming Lecture 20's Strong Duality Theorem holds here even though f is not quadratic and admits no closed-form dual function. Notice also $q(0) = 2953.2359$ — the unconstrained optimum from Exercise 1 — exactly as expected, since $\lambda = 0$ relaxes the SLA constraint entirely.

1.6 Take-Away Exercises

Work through each exercise following these steps: 1. Formulate the NLP (decision variables, objective, constraint if any). 2. Derive the gradient (and, where asked, the constraint gradient). 3. Solve using Python SciPy (`scipy.optimize.minimize`). 4. Where indicated, carry out the sensitivity or duality analysis requested. 5. Record the optimal solution and write a one-paragraph managerial interpretation.

1.6.1 Exercise 1 — Unconstrained

BMTC wants to site a new satellite bus depot in Bengaluru. The depot will receive new buses from two manufacturer delivery yards and dispatch them across three service terminals (coordinates in km, local grid):

Node	Type	x (km)	y (km)	Flow (buses/week)
Manufacturer Yard 1	Upstream	5	55	25
Manufacturer Yard 2	Upstream	70	60	35
Service Terminal 1	Downstream	15	10	20
Service Terminal 2	Downstream	45	25	18
Service Terminal 3	Downstream	75	15	22

Formulate and solve for the depot location minimizing total flow-weighted distance. Use `scipy.optimize.minimize` with `method='BFGS'`, supplying the analytic gradient.

1.6.2 Exercise 2 — Constrained, with Sensitivity Analysis

NHAI wants to site a highway incident-response dispatch yard. The yard draws its crew and equipment from a single depot and must be able to reach three accident-prone stretches (coordinates in km):

Node	Type	x (km)	y (km)	Flow-equivalent weight
Crew Depot	Upstream	10	10	10
Stretch A	Downstream	30	60	12
Stretch B	Downstream	70	40	15
Stretch C	Downstream	90	80	10

A response-time SLA requires the yard to lie within $R = 35$ km of Stretch C, the most accident-prone of the three. Formulate the constrained NLP, solve it with `method='trust-constr'`, and report: 1. The optimal site and the KKT multiplier λ^* (with respect to $b = R^2$). 2. The direct sensitivity $\lambda_R = -\partial f^*/\partial R$, and the daily/weekly value (in weight-km per km) of relaxing the SLA radius by 1 km.

1.6.3 Exercise 3 — Constrained (Exclusion Zone)

Delhivery wants to site a new cross-dock in Pune, serving the same two upstream plants and three downstream clusters as this lecture's In-Class Exercises. However, municipal zoning requires the facility to remain at least 8 km from an eco-sensitive buffer zone centered at $(40, 30)$.

1. Formulate the exclusion constraint $g(x, y) = R^2 - \text{dist}(x, y)^2 \leq 0$ with $(x_0, y_0) = (40, 30)$, $R = 8$.

2. Verify that the Exercise 1 unconstrained optimum (41.55, 30.21) violates this constraint (it lies inside the buffer).
 3. Solve the constrained problem with `method='trust-constr'` and report the KKT multiplier. Is its sign consistent with KKT dual feasibility ($\lambda^* \geq 0$) despite the constraint pointing “outward” rather than “inward”? Explain why.
-

1.6.4 Exercise 4 — Duality Verification (Python Implementation)

Using Exercise 2’s NHA1 dispatch-yard problem: 1. Construct the Lagrangian $\mathcal{L}(x, y, \lambda) = f(x, y) + \lambda g(x, y)$ and its gradient. 2. Implement $q(\lambda) = \inf_{x, y} \mathcal{L}(x, y, \lambda)$ in Python by minimizing $\mathcal{L}$ over (x, y) for a grid of fixed $\lambda \geq 0$ values (as done in this lecture’s Bonus section). 3. Plot $q(\lambda)$ using `matplotlib`, mark the λ^* found by `trust-constr` in Exercise 2, and confirm $q(\lambda^*) = f^*$ from the primal solve — verifying strong duality. 4. Write a 2-3 sentence note connecting this to Lecture 20’s Property 2 (weak duality: $q(\lambda) \leq f^*$ for every dual-feasible λ) — check that every other λ you plotted indeed gives $q(\lambda) < f^*$.

1.7 Circling Back

- Lecture 12 (NLP Principles): the facility-location problem was previewed there as Example 2, and this lecture is where that preview becomes a fully worked, two-sided application.
- Lectures 13-15 (Gradient Descent, Newton’s Method, BFGS): all three solved the unconstrained Weber problem to the same global optimum (guaranteed by convexity), differing only in iterations and function evaluations needed — reinforcing that method choice is about efficiency, not correctness, once convexity is established.
- Lectures 16-18 (Penalty, Barrier, Interior-Point): the constrained facility-location problem is solved directly with Interior-Point via `trust-constr`, which both handles the non-linear constraint and recovers the KKT multiplier as a by-product — exactly the workflow these lectures built.
- Lecture 19 (NLP Sensitivity Analysis): this lecture is the direct answer to Lecture 19’s own forward reference — “how much would relaxing a demand-coverage requirement save?” — with the added twist that, unlike the quadratic examples there, the shadow price itself varies continuously with the resource level, since f is convex but not quadratic.
- Lecture 20 (NLP Duality): strong duality is verified numerically rather than symbolically, since the Weber objective has no closed-form Lagrangian dual — a realistic illustration of why the algorithms in Lectures 16-18, not hand analysis, are the primary tool for non-linear transportation problems.
- Lecture 11 (Transshipment Problem): contrasted throughout — there the hub location was fixed and flow was the decision; here location is the decision and flow is fixed, the natural next step in generality.

1.8 Moving Forward

- Lecture 22 (Transportation Engineering Problems - IV): extends the non-linear toolkit to network-flow problems where *cost itself* depends on flow — congestion. The linear flow-conservation networks of Lectures 10-11 are revisited with non-linear, flow-dependent arc costs, closing the arc that began with the Maximum Flow Problem.

1.9 Further Reading

- Drezner, Z. and Hamacher, H.W. (Eds.) (2004). *Facility Location: Applications and Theory*. Springer — Chapter 1 (the Weber problem and its history).
- Love, R.F., Morris, J.G., and Wesolowsky, G.O. (1988). *Facility Location: Models and Methods*. North-Holland — continuous location models, weighted-distance objectives.
- Nocedal, J. and Wright, S.J. (2006). *Numerical Optimization* (2nd ed.). Springer — Chapter 12 (constrained optimization, sensitivity and active-set behaviour).
- Boyd, S. and Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press — Chapter 5 (Lagrangian duality). Freely available at web.stanford.edu/~boyd/cvxbook.
- SciPy documentation: `scipy.optimize.minimize` with `method='trust-constr'` and `NonlinearConstraint` — docs.scipy.org